

how the size changes each time new items are allocated. Let's check the memory allocated currently:

```
# check the memory allocated
import sys
print(sys.getsizeof(list1))
```

The output is:

96

Here is a common function to see how much memory is allocated before and after values are appended:

```
# appending the new item
def append_into_list(value):
    print("address: ", id(list1))
    print("before size of list: ", sys.getsizeof(list1))
    list1.append(value)
    print("updated list: ", list1)
    print("address remains the same: ", id(list1))
    print("after sizes of list: ", sys.getsizeof(list1))
    print("")
```

Please closely observe the size and memory address of the list before and post update.

 **Note:** In the scenario given below, the memory address didn't change, but this will not always be the case.

```
# lets see after adding couple of more values the list size
append_into_list(6)
append_into_list(7)
append_into_list(8)
append_into_list(9)
append_into_list(10)
append_into_list(11)
append_into_list(12)
```

The output is:

```
address: 140509666477824
before size of list: 96
updated list: [100, 2, 'three', 4, 5, 6]
address remains the same: 140509666477824
after size of list: 128
```

```
address: 140509666477824
before size of list: 128
updated list: [100, 2, 'three', 4, 5, 6, 7]
address remains the same: 140509666477824
```

```
after size of list: 128
```

```
address: 140509666477824
before size of list: 128
updated list: [100, 2, 'three', 4, 5, 6, 7, 8]
address remains the same: 140509666477824
after size of list: 128
```

```
address: 140509666477824
before size of list: 128
updated list: [100, 2, 'three', 4, 5, 6, 7, 8, 9]
address remains the same: 140509666477824
after size of list: 128
```

```
address: 140509666477824
before size of list: 128
updated list: [100, 2, 'three', 4, 5, 6, 7, 8, 9, 10]
address remains the same: 140509666477824
after size of list: 192
```

```
address: 140509666477824
before size of list: 192
updated list: [100, 2, 'three', 4, 5, 6, 7, 8, 9, 10, 11]
address remains the same: 140509666477824
after size of list: 192
```

```
address: 140509666477824
before size of list: 192
updated list: [100, 2, 'three', 4, 5, 6, 7, 8, 9, 10, 11, 12]
address remains the same: 140509666477824
after size of list: 192
```

As you can see, the size of the list first expanded from 96 to 128, but didn't change for the next couple of items and stayed there for some time. Then the size expanded to 192. The reason is that in CPython the memory is preallocated in chunks beforehand. This is to avoid making frequent heavy system calls. And if you see, the allocation is not static but mild and linear.

The reallocation happens to extend the current memory needed. So when you have a huge array in need and the realloc does not have so much space, it will create new memory and copy; this will be a very expensive operation. To avoid this, we can preallocate the required memory. The code snippet of C implementation of list is given below. The pictorial representation is given in Figure 1.

```
#### Cpython : https://github.com/python/cpython/blob/master/
Objects/listobject.c
'''
    /* This over-allocates proportional to the list size,
```